Coherence and consistency

(reference: text Section 5.10)

in multiprocessor systems need to ensure that the memory system provides coherence and consistency for parallel programs

coherence

essentially, results of executing a parallel program should be *the same as if there was just one main memory and no caches*

- constrains possible results from accesses to a single memory block

example:

X is initially 0

core 1	core 2	core 3	core 4
X ← 1	X ← 2	read X	read X
		read X	read X

don't have coherence if, e.g., on core 3 the value of X could be read as "1" and then as "2", and yet on core 4 read as "2" and then as "1"

consistency

with the strongest type of consistency ("sequential consistency"), the results of executing a parallel program *must be the same as an execution in which operations occur sequentially in some order consistent with program order*

... but commonly processors implement a slightly weaker notion of consistency

example:

X and Y are both initially 0

core 1	core 2
X ← 1	Y ← 1
read Y	read X

typically, because of use of write buffers, both of these read operations *could* return "0"!

how can we ensure sequentially consistent behavior, *when needed*, for parallel programs?

... instruction set support, e.g. "fence" instruction, "aq"/"rl" options, in RISC-V

implementing coherence

can achieve coherence if:

- on a read miss at a processor, if the required memory block is dirty in another processor's cache, get the updated memory block
- when write to a memory block, first read block into cache if not present (getting updated copy if dirty in another cache), and then invalidate all copies in the caches of other processors

MESI ("Modified, Exclusive, Shared, Invalid") cache coherence protocol (one of a number of such protocols)

four possible states for a "line" (memory block in our terminology) at the cache of a particular processor core:

modified state: only this cache has a copy, which is dirty

exclusive state: only this cache has a copy, which is clean

shared state: has clean copy, may also be in other caches

invalid state: invalidated or not present

given these four states, what state transitions occur on local and remote reads and writes?

hmmm … suppose read miss, for example … how does *the core at which the miss occurred* know where to get the data from?

two basic implementation approaches: snooping protocols (*simpler*) and directory protocols (*more scalable*) ("snoop filters" are hybrid approaches)

snooping protocols

> on a read miss at a core, *broadcast* a message that will be seen by all cores
>
> when write, may similarly need to *broadcast*

directory protocols

> use a directory, implemented in hardware, to keep track of the caching status of each block … now can consult the directory to determine *exactly where* messages need to be sent!

false sharing

occurs when threads running on different cores are reading & writing different variables that happen to occupy the same memory block

false sharing can greatly hurt performance (… *why*?)